OneRec-V2 Technical Report (2025)

Challenges

- **Compute is spent on context encoding, not the loss-relevant generation.** In OneRec-V1's encoder–decoder setup, long user context dominates FLOPs; the paper highlights that with context length 512, *"the context encoding consumes 97.66% of total FLOPs"* while decoding is only 2.34%.
- **Scaling bottleneck from architecture imbalance.** Under fixed compute, the encoder-heavy workload limits scaling the generative component where recommendation decisions are made.
- **Reward-model-only RL issues.** Reward-model RL can be sample-inefficient (extra rollout+scoring cost) and vulnerable to reward hacking due to proxy rewards.
- **Using real user feedback is non-trivial.** Raw watch time is biased by video duration; multiple objectives and sparse positives complicate RL from feedback.

Key Initiatives

- **Lazy Decoder-Only Architecture.** Remove the encoder and treat user context as static conditioning accessed via cross-attention, eliminating redundant context computation.
- **Lazy cross-attention without K/V projections + GQA.** Context processor outputs (layer-shared) key/value tensors; decoder blocks apply cross-attention without key/value projection layers and use Grouped Query Attention to reduce memory/IO.
- **Scaling to large models under the same compute.** Reported *"94% reduction in computational requirements"* and *"90% reduction in actual training resources"*, enabling scaling up to 8B parameters and observing smooth scaling behavior.
- **Preference alignment from real-world user interactions.** (i) Duration-Aware Reward Shaping to debias watch-time signals; (ii) Gradient-Bounded Policy Optimization (GBPO) with adaptive ratio bounding to stabilize RL.

Methodology

- **Input/target tokenization.** Each item is mapped by a semantic tokenizer to 3 semantic IDs; training uses input `[BOS, s_1, s_2]` and predicts `[s_1, s_2, s_3]` via autoregressive generation.
- **Context Processor.** Heterogeneous user features (profile + behaviors across pathways) are concatenated into a context sequence and transformed into normalized per-layer key/value pairs $\{(k_0, v_0), \ldots\}$, with configurable key/value sharing across decoder blocks (parameters L_{kv}, S_{kv}).
- **Lazy Decoder Blocks.** Stack of transformer blocks with: (1) lazy cross-attention to context K/V (often shared across blocks); (2) causal self-attention over the short semantic-token sequence; (3) FFN (with optional MoE in deeper layers).
- **Compute allocation insight.** For lazy decoder-only, target decoding becomes $\approx 100\%$ of compute regardless of context length (Table 1), versus $< 3\%$ for classic enc–dec/naive dec-only at context length 512.
- **Duration-Aware Reward Shaping.** Bucket each user's historical videos by log-duration $F(d) = \lfloor \log_\beta(d+\epsilon) \rfloor$. For target video i, compute percentile q_i of its playtime within the corresponding duration bucket, then select positives as top-quartile (and explicit "dislike" as negatives).
- **GBPO.** A policy-optimization objective designed to (a) keep gradients from all samples (no ratio clipping drop) while (b) bounding gradients for negative samples with a dynamic bound related to token probability, empirically stabilizing gradient norms versus ECPO/GRPO.

Experiments

- **Architecture efficiency vs loss.** Across 0.1B/0.5B/1B scales, lazy decoder-only matches convergence loss of enc–dec while using far fewer FLOPs and activations (e.g., at 1B: 18.89 GFLOPs and 1.24B activations vs 296.36 GFLOPs and 17.63B activations for enc:dec=1:1; Table 2).
- **Scaling law behavior.** Dense lazy-decoder models from 0.1B→8B show decreasing convergence loss 3.57→3.19; fitted scaling law $\hat{L}(N) = E + A/N^{\alpha}$ with $E = 3.13, A = 3660, \alpha = 0.489$ (Figure 6).
- **Sparse MoE variant.** 4B total (0.5B active) MoE reaches convergence loss 3.22, outperforming 2B dense while keeping compute similar to 0.5B dense (Table 5 discussion).
- **User-feedback RL (online).** Duration-aware feedback RL improves App Stay Time; adding OneRec-generated samples (on-policy) improves most interaction metrics too (Table 6; e.g., Kuaishou App Stay Time +0.227%, Video View +0.716%).
- **Reward model vs user feedback vs hybrid.** Reward model tends to boost interaction metrics more; user feedback tends to boost App Stay Time more; hybrid improves balance (Table 7).
- **Full online A/B test.** 5% traffic, 1 week; 1B params, context length 3000, beam size 512; L20 GPUs with 36ms latency and MFU 62%. Reported overall improvements vs OneRec-V1: App Stay Time *"+0.467%"* (Kuaishou) and *"+0.741%"* (Kuaishou Lite), with broad gains in interaction metrics (Table 8).

References (selected)

- Zhou et al., *OneRec Technical Report*, 2025 (OneRec-V1 baseline).
- Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022 (scaling law).
- Ainslie et al., *GQA: Training Generalized Multi-Query Transformer Models*, 2023 (Grouped Query Attention).
- Schulman et al., *Proximal Policy Optimization Algorithms*, 2017 (PPO background).
- Shao et al., *DeepSeek-Math*, 2024 (GRPO reference in paper).
- Liu et al., *DeepSeek-V3 Technical Report*, 2024 (aux-loss-free MoE load balancing reference).

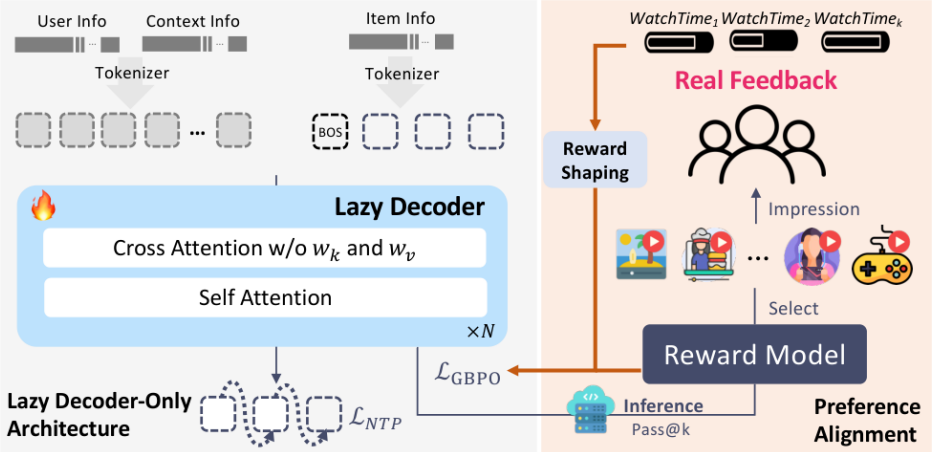

Figure 2 | The overall architecture and post-training framework of OneRec-V2. The left panel illustrates the Lazy Decoder-Only Architecture, The right panel depicts the post-training preference alignment process

.

wide range of model sizes. This provides both empirical and theoretical guidance for the future development of large generative recommendation models.

2. **Preference Alignment with Real-World User Interactions**. We introduce a comprehensive post-training framework that directly leverages real-world user feedback signals to address the fundamental challenges of reward modeling in generative recommender systems. (i) *Duration-Aware Reward Shaping*, which mitigates the inherent bias in raw watch time signals by accounting for video length variations, ensuring that reward signals accurately reflect content quality rather than merely duration; and (ii) *Adaptive Ratio Clipping*, which effectively reduces training variance while preserving convergence guarantees in the policy optimization process. Our experiments demonstrate significant gains in *APP Stay Time*. Notably, we observe amplified online performance when incorporating traffic distribution patterns from OneRec's own recommendations, suggesting improved alignment between model optimization and real-world user behavior distributions.

Extensive online A/B testing on Kuaishou/Kuaishou Lite APP with 400 million daily active users demonstrates that OneRec-V2 achieves significant improvements compared to OneRec-V1, delivering improvements of 0.467% and 0.741% in App Stay Time, while effectively balancing multiple recommendation objectives without seesaw effects.

In the remainder of this paper, we first elaborate on the OneRec-V2 architecture and empirical results of pre-training (Section 2). Next, we present post-training method (Section 3), followed by a comprehensive evaluation through online A/B testing (Section 4). Finally, we conclude this work with a discussion of existing limitations and propose potential directions for future research (Section 5).

Figure 1: Overall architecture and post-training framework of OneRec-V2 (paper Figure 2).